

Informe de Entrega — Proyecto Integrador (Sesión 10)

MOD2: Programación de Backend y MCP en Python para IA Generativa | CEDIA - UNIANDES

Estudiante: Juan Carlos Lozada	Institución: Universidad Regional Autónoma de Los Andes (UNIANDES)
Fecha: 18 de septiembre de 2026	Dominio: Sistema de Gestión y Reserva de Espacios (Consultorios, Salas, Canchas)
Metodología: Spec-Driven Development (Spec-Kit + Antigravity)	Repositorio: https://github.com/jclozadacastillo/curso-mcp

Resumen Ejecutivo y Métricas Clave

El presente proyecto integrador implementa un sistema backend de producción y un servidor bajo el **Model Context Protocol (MCP)** para la administración y reserva de espacios compartidos. El desarrollo fue guiado estrictamente por la metodología **Spec-Driven Development (SDD)** con Spec-Kit, estableciendo 4 artefactos formales (constitution.md, spec.md, plan.md, tasks.md) antes de escribir código. El sistema cuenta con dualidad funcional completa: **API REST con FastAPI** y **Servidor MCP con FastMCP** reutilizando el 100% de la capa de servicios.

Pruebas Automatizadas 72 / 72 PASSED 100% aprobación	Cobertura Services 100.0% Umbral: ≥90%	Cobertura Global 94.0% Umbral: ≥70%	Principio DIP 0% Mocks Fake Repo inyectado
--	--	---	--

1. Justificación Arquitectónica (20% - Nivel Excelente)

El proyecto sigue un patrón de monolito modular con separación estricta de responsabilidades bajo app/. Se aplica el **Principio de Inversión de Dependencias (DIP)** donde services/ recibe el repositorio como parámetro con valor por defecto (repo=reserva_repository), garantizando desacoplamiento absoluto y posibilitando pruebas unitarias puras con RepositorioFalsoReservas sin recurrir a unittest.mock.

Regla de Negocio / Decisión	Capa Donde Vive	Justificación en Vivo (Por qué vive ahí)
Anti-Solapamiento (max(I1,I2) < min(F1,F2))	app/services/reservas.py	Es una regla de negocio central pura. Si estuviera en routers, MCP la duplicaría. Si estuviera en repositorios, se acoplaría la persistencia a lógica algorítmica.
Borde Exacto Contiguo (10:00=10:00 permitido)	app/services/reservas.py	Aclarado formalmente en spec.md vía /speckit.clarify. Vive en la función pura _hay_solapamiento aislada y determinista.
Titularidad y Aislamiento (usuario_id == owner)	app/services/reservas.py	Protege contra vulnerabilidades BOLA/IDOR. Lanza PermisoDenegadoError que el router traduce a HTTP 403 y MCP a error estructurado.
Confirmación Destructiva (confirmacion=True obligatoria)	app/services/reservas.py	La seguridad de acciones destructivas debe vivir en el servidor, jamás dependiendo de que el cliente o el modelo de lenguaje 'decida preguntar'.

2. Seguridad y Gestión de Identidad (20% - Nivel Excelente)

Autenticación y Criptografía: Contraseñas hasheadas con bcrypt (vía passlib[bcrypt]). Autenticación mediante OAuth2 Password Flow con JWT firmados criptográficamente (HS256) con expiración a 30 minutos. Secretos leídos desde .env mediante pydantic-settings con plantilla .env.example versionada.

Identidad Inmutable: Ningún endpoint acepta usuario_id del cliente. La identidad se obtiene exclusivamente desde el JWT decodificado en la dependencia get_current_user.

¿Qué pasaría si un endpoint no verificara el JWT? Cualquier atacante podría enviar usuario_id arbitrarios en el payload, logrando acceso no autorizado a información confidencial de otros usuarios, cancelando reservas ajenas (ataque BOLA/IDOR) y destruyendo la trazabilidad y el no-repudio en las auditorías del sistema.

3. REST y Servidor MCP (25% - Nivel Excelente)

La aplicación expone una API REST semántica completa y 4 tools MCP construidas con FastMCP. Ambos canales reutilizan la misma capa services/reservas.py sin duplicación de código.

Canal / Herramienta	Firma / Endpoint	Comportamiento y Contrato de Error
REST: Crear	POST /reservas/ (201)	Valida JWT, horas y solapamiento. Retorna ReservaResponse. Errores: 400 solapamiento, 401 sin token, 422 formato.
REST: Cancelar	DELETE /reservas/{id} (204)	Exige confirmacion=true y titularidad. Retorna 204 No Content. Errores: 400 sin confirmar, 403 no dueño, 404 no existe.
MCP: crear_reserva	crear_reserva(espacio, fecha, hora_inicio, hora_fin, motivo)	Reutiliza services.reservas.crear_reserva(). Retorna dict estructurado o {"error": "Conflicto de horario..."}

MCP: listar_reservas	listar_reservas(skip, limit, fecha)	Retorna reservas propias con paginación y filtro de fecha opcional. Maneja identidad de usuario.
MCP: consultar_disp	consultar_disponibilidad(espacio, fecha)	Retorna total ocupado y bloques de horario ya tomados para planificar agendas.
MCP: cancelar_reserva	cancelar_reserva(reserva_id, confirmacion)	Exige confirmación del servidor (confirmacion=True). Retorna confirmación o error de falta de permiso.

4. Testing y Certificación de Cobertura (20% - Nivel Excelente)

Se aplicó una pirámide de pruebas completa: 16 pruebas unitarias con DIP, 11 pruebas de API REST, 9 pruebas MCP y prueba de integración con base de datos SQLite real. Cada regla de negocio de spec.md cuenta con su test identificable.

Módulo / Capa	Sentencias	Líneas Perdidas	Cobertura	Estado Constitucional
app/services/reservas.py	69	0	100.0%	CUMPLE (Exige ≥90%)
app/services/gastos.py	49	0	100.0%	CUMPLE
app/services/usuarios.py	24	0	100.0%	CUMPLE
app/repositories/reservas.py	32	1	97.0%	CUMPLE
app/routers/reservas.py	44	2	95.0%	CUMPLE
app/utills/security.py	22	0	100.0%	CUMPLE
TOTAL GLOBAL PROYECTO	623	35	94.0%	CUMPLE (Exige ≥70%)

5. Spec-Driven Development y Corrección al Agente (15% - Nivel Excelente)

Los 4 Artefactos Encadenados: Ubicados en specs/002-sistema-reservas/ (constitution.md, spec.md, plan.md, tasks.md).
Momento Real de Corrección al Agente (Registrado en analysis.md): Durante la fase preliminar del plan de implementación, el agente propuso ejecutar la eliminación de reservas directamente en el router invocando el repositorio sin pasar por services/, omitiendo validar si la reserva pertenecía al usuario emisor del JWT y sin requerir confirmación en el servidor. Se intervino al agente exigiendo el cumplimiento de los Artículos I.2, III.3, IV.3 y VI.4, obligándolo a: (1) trasladar la lógica a services.reservas.cancelar_reserva() con DIP, (2) validar la propiedad del recurso lanzando PermisoDenegadoError (403), y (3) establecer la bandera obligatoria confirmacion: bool = False en el servidor lanzando AccionNoConfirmadaError (400).

6. Guía Rápida para la Demostración en Vivo (Sesión 10)

- API REST (Swagger):** Iniciar servidor con .venv\Scripts\uvicorn.exe app.main:app --reload y abrir http://127.0.0.1:8000/docs. Autenticar vía /usuarios/token y demostrar creación de reservas, rechazo por solapamiento (400), rechazo por falta de confirmación al borrar y borrado exitoso (204).
- MCP Inspector:** Ejecutar npx @modelcontextprotocol/inspector .venv\Scripts\python.exe -m app.mcp.server. Ejecutar consultar_disponibilidad y crear_reserva demostrando que devuelve {"error": "..."} controlado ante conflictos.
- Pytest y Cobertura:** Ejecutar .venv\Scripts\pytest.exe --cov=app --cov-report=term-missing evidenciando **72 tests aprobados** y **94% de cobertura global** (100% en servicios).